PYTHON CASE STUDIES – 10 PROGRAMS

Separate algorithms, Python code, flowcharts, and sample outputs

1. University Scholarship Allocation

A. Algorithm

1. Start.
2. Input the number of students.
3. For each student, input name, five subject marks, attendance, and family income.
4. Calculate aggregate percentage.
5. Check aggregate >= 75, every subject >= 40, attendance >= 75, and income <= 300000.
6. Store eligible students.
7. Rank by aggregate descending; use attendance descending to break ties.
8. Display ranking and stop.

B. Python Code

```
students = []
n = int(input("Enter number of students: "))

for i in range(n):
    print("\nStudent", i + 1)
    name = input("Enter name: ")
    marks = list(map(int, input("Enter 5 subject marks: ").split()))
    attendance = float(input("Enter attendance percentage: "))
    income = float(input("Enter family income: "))
    aggregate = sum(marks) / len(marks)

    if (aggregate >= 75 and min(marks) >= 40
            and attendance >= 75 and income <= 300000):
        students.append([name, aggregate, attendance])
        print("Eligible for scholarship")
    else:
        print("Not eligible for scholarship")

students.sort(key=lambda x: (x[1], x[2]), reverse=True)
print("\nSCHOLARSHIP RANKING")
for rank, student in enumerate(students, 1):
    print(rank, student[0], "Aggregate:", round(student[1], 2),
          "Attendance:", student[2])
```

C. Flowchart (text to copy and draw)

```
START
  |
Input number of students
  |
Input student details
  |
Calculate aggregate marks
  |
Check all eligibility conditions
  |
Eligible? --No--> Display Not Eligible
  |
 Yes
  v
Store eligible student
  |
More students? --Yes--> Input next student
  |
 No
  v
Sort by aggregate, then attendance
  |
Display scholarship ranking
  |
STOP
```

D. Sample Output

```
Enter number of students: 2

Student 1
Enter name: Ravi
Enter 5 subject marks: 85 90 80 75 88
Enter attendance percentage: 90
```

```
Enter family income: 200000
Eligible for scholarship

Student 2
Enter name: Priya
Enter 5 subject marks: 70 65 80 75 60
Enter attendance percentage: 85
Enter family income: 250000
Not eligible for scholarship

SCHOLARSHIP RANKING
1 Ravi Aggregate: 83.6 Attendance: 90.0
```

2. Progressive Electricity Billing

A. Algorithm

1. Start and input the number of months.
2. For each month, input units consumed.
3. Charge first 100 units at Rs. 2/unit, next 100 at Rs. 3/unit, and remaining units at Rs. 5/unit.
4. Add fixed charge of Rs. 50.
5. Add 5% surcharge if units exceed 300; apply 5% rebate on energy charge if units are below 100.
6. Store and display each bill.
7. Compare consecutive bills; flag an increase above 30%.
8. Stop.

B. Python Code

```
bills = []
n = int(input("Enter number of months: "))

for i in range(n):
    print("\nMonth", i + 1)
    units = float(input("Enter units consumed: "))

    if units <= 100:
        charge = units * 2
    elif units <= 200:
        charge = 100 * 2 + (units - 100) * 3
    else:
        charge = 100 * 2 + 100 * 3 + (units - 200) * 5

    surcharge = charge * 0.05 if units > 300 else 0
    rebate = charge * 0.05 if units < 100 else 0
    total = charge + 50 + surcharge - rebate
    bills.append(total)
    print("Electricity Bill: Rs.", round(total, 2))

print("\nMONTHLY BILL COMPARISON")
for i, bill in enumerate(bills):
    print("Month", i + 1, ":", round(bill, 2))
    if i > 0:
        increase = ((bill - bills[i-1]) / bills[i-1]) * 100
        if increase > 30:
            print("Warning: Unusual increase detected!")
        else:
            print("Normal bill variation")
```

C. Flowchart (text to copy and draw)

```
START
  |
Input number of months
  |
Input monthly units
  |
Calculate progressive slab charge
  |
Add fixed charge
  |
Apply surcharge/rebate if applicable
  |
Calculate and store bill
  |
More months? --Yes--> Repeat
  |
 No
  v
Compare consecutive bills
  |
Increase above 30%?
  |Yes: Warning | No: Normal
  v
Display monthly bills
  |
STOP
```

D. Sample Output

```
Enter number of months: 3
Month 1
Enter units consumed: 80
Electricity Bill: Rs. 202.0
Month 2
Enter units consumed: 150
```

```
Electricity Bill: Rs. 400.0
Month 3
Enter units consumed: 250
Electricity Bill: Rs. 900.0

MONTHLY BILL COMPARISON
Month 1 : 202.0
Month 2 : 400.0
Warning: Unusual increase detected!
Month 3 : 900.0
Warning: Unusual increase detected!
```

3. Hospital Billing and Patient Classification

A. Algorithm

1. Start and input patient name, consultation, diagnostic, and treatment charges.
2. Calculate total charges.
3. Select patient category: senior citizen (10%), concession card (15%), or normal (0%).
4. Calculate concession and final bill.
5. Input amount paid and validate that it is between zero and the final bill.
6. Calculate balance.
7. Classify as Fully Paid, Partially Paid, or Unpaid.
8. Display bill and stop.

B. Python Code

```
name = input("Enter patient name: ")
consultation = float(input("Consultation charge: "))
diagnostic = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))
total = consultation + diagnostic + treatment

print("1. Senior Citizen\n2. Concession Card\n3. Normal Patient")
choice = int(input("Select patient category: "))

if choice == 1:
    discount = total * 0.10
elif choice == 2:
    discount = total * 0.15
elif choice == 3:
    discount = 0
else:
    print("Invalid patient category")
    raise SystemExit

final_bill = total - discount
print("Total Charges:", total)
print("Concession:", discount)
print("Final Bill:", final_bill)

paid = float(input("Enter amount paid: "))
if paid < 0 or paid > final_bill:
    print("Invalid payment amount")
else:
    balance = final_bill - paid
    if balance == 0:
        status = "Fully Paid"
    elif paid > 0:
        status = "Partially Paid"
    else:
        status = "Unpaid"

    print("\nHOSPITAL BILL")
    print("Patient:", name)
    print("Final Bill:", final_bill)
    print("Amount Paid:", paid)
    print("Balance:", balance)
    print("Classification:", status)
```

C. Flowchart (text to copy and draw)

```
START
  |
Input patient and charge details
  |
Calculate total charges
  |
Select category and apply concession
  |
Calculate final bill
  |
Input payment
  |
Payment valid? --No--> Display error
  |
 Yes
  v
Calculate balance and classify patient
  |
Display hospital bill
  |
STOP
```

D. Sample Output

```
Enter patient name: Rahul
Consultation charge: 500
Diagnostic charge: 1000
Treatment charge: 3000
1. Senior Citizen
2. Concession Card
3. Normal Patient
Select patient category: 1
Total Charges: 4500.0
Concession: 450.0
Final Bill: 4050.0
Enter amount paid: 3000

HOSPITAL BILL
Patient: Rahul
Final Bill: 4050.0
Amount Paid: 3000.0
Balance: 1050.0
Classification: Partially Paid
```

4. Employee Payroll and Deduction Analysis

A. Algorithm

1. Start and input employee name, basic salary, allowances, overtime, and incentives.
2. Calculate gross salary.
3. Apply progressive deduction: 0% on first Rs. 10,000; 5% on next Rs. 10,000; 10% above Rs. 20,000.
4. Calculate net salary = gross salary - deduction.
5. Display formatted salary statement and stop.

B. Python Code

```
name = input("Enter employee name: ")
basic = float(input("Enter basic salary: "))
allowances = float(input("Enter allowances: "))
overtime = float(input("Enter overtime amount: "))
incentives = float(input("Enter incentives: "))

gross = basic + allowances + overtime + incentives

if gross <= 10000:
    deduction = 0
elif gross <= 20000:
    deduction = (gross - 10000) * 0.05
else:
    deduction = 10000 * 0.05 + (gross - 20000) * 0.10

net = gross - deduction

print("\nEMPLOYEE SALARY STATEMENT")
print("--------------------------------")
print("Employee Name:", name)
print("Basic Salary:", basic)
print("Allowances:", allowances)
print("Overtime:", overtime)
print("Incentives:", incentives)
print("Gross Salary:", gross)
print("Deductions:", deduction)
print("Net Salary:", net)
```

C. Flowchart (text to copy and draw)

```
START
  |
Input employee details
  |
Calculate gross salary
  |
Determine progressive deduction
  |
Calculate net salary
  |
Generate salary statement
  |
STOP
```

D. Sample Output

```
Enter employee name: Ravi
Enter basic salary: 25000
Enter allowances: 5000
Enter overtime amount: 2000
Enter incentives: 3000

EMPLOYEE SALARY STATEMENT
--------------------------------
Employee Name: Ravi
Basic Salary: 25000.0
Allowances: 5000.0
Overtime: 2000.0
Incentives: 3000.0
Gross Salary: 35000.0
Deductions: 2000.0
Net Salary: 33000.0
```

5. Multi-Plan Telecommunications Cost Optimization

A. Algorithm

1. Start and input data usage, call minutes, required validity, and budget.
2. Input number of plans and each plan's data limit, call limit, validity, and cost.
3. Calculate excess data charge at Rs. 20/GB and excess call charge at Rs. 0.50/minute.
4. Calculate total plan cost including excess charges.
5. Keep plans that meet validity and budget.
6. Select the eligible plan with the lowest total cost.
7. Display the best plan or report that none is eligible. Stop.

B. Python Code

```
data_used = float(input("Enter data used in GB: "))
calls_used = int(input("Enter call minutes used: "))
required_days = int(input("Enter required validity: "))
budget = float(input("Enter budget: "))
n = int(input("Enter number of plans: "))

plans = []
for i in range(n):
    print("\nPlan", i + 1)
    name = input("Plan name: ")
    data = float(input("Data limit in GB: "))
    calls = int(input("Call limit in minutes: "))
    validity = int(input("Validity in days: "))
    cost = float(input("Plan cost: "))

    extra_data = max(0, data_used - data)
    extra_calls = max(0, calls_used - calls)
    total = cost + extra_data * 20 + extra_calls * 0.5
    print("Total cost:", total)

    if validity >= required_days and total <= budget:
        plans.append([name, total])

if plans:
    best = min(plans, key=lambda x: x[1])
    print("\nMOST ECONOMICAL PLAN")
    print("Plan:", best[0])
    print("Total Cost:", best[1])
else:
    print("\nNo eligible plan within budget")
```

C. Flowchart (text to copy and draw)

```
START
  |
Input customer usage and budget
  |
Input plan details
  |
Calculate excess data/call charges
  |
Calculate total plan cost
  |
Meets validity and budget?
  |Yes: Store plan | No: Skip
  |
More plans? --Yes--> Repeat
  |
 No
  v
Any eligible plan?
  |Yes: Find lowest cost
  |No: Display no eligible plan
  v
Display result
  |
STOP
```

D. Sample Output

```
Enter data used in GB: 15
Enter call minutes used: 500
Enter required validity: 28
Enter budget: 400
Enter number of plans: 2

Plan 1
Plan name: Basic
```

```
Data limit in GB: 10
Call limit in minutes: 400
Validity in days: 28
Plan cost: 200
Total cost: 450.0

Plan 2
Plan name: Premium
Data limit in GB: 20
Call limit in minutes: 600
Validity in days: 56
Plan cost: 350
Total cost: 350.0

MOST ECONOMICAL PLAN
Plan: Premium
Total Cost: 350.0
```

6. Railway Fare and Reservation Calculator

A. Algorithm

1. Start and input number of passengers.
2. For each passenger, input name, age, distance, and travel class.
3. Calculate distance fare: first 100 km at Rs. 1/km; next 200 km at Rs. 0.80/km; above 300 km at Rs. 0.60/km.
4. Apply class multiplier: Sleeper 1, AC 2, First Class 3.
5. Children below 5 travel free; passengers aged 60 or above receive 40% concession.
6. Add Rs. 50 reservation fee for passengers aged 5 or above.
7. Ask whether ticket is cancelled. If yes, deduct 20% penalty and display refund.
8. Display fare and stop.

B. Python Code

```
n = int(input("Enter number of passengers: "))

for i in range(n):
    print("\nPassenger", i + 1)
    name = input("Enter passenger name: ")
    age = int(input("Enter age: "))
    distance = float(input("Enter distance in km: "))

    print("1. Sleeper\n2. AC\n3. First Class")
    choice = int(input("Select class: "))

    if distance <= 100:
        fare = distance * 1
    elif distance <= 300:
        fare = 100 + (distance - 100) * 0.8
    else:
        fare = 100 + 200 * 0.8 + (distance - 300) * 0.6

    if choice == 1:
        fare *= 1
    elif choice == 2:
        fare *= 2
    elif choice == 3:
        fare *= 3
    else:
        print("Invalid class")
        continue

    if age < 5:
        fare = 0
        reservation = 0
    else:
        if age >= 60:
            fare *= 0.60
        reservation = 50

    total = fare + reservation
    print("\nPassenger:", name)
    print("Ticket Fare:", round(total, 2))

    cancel = input("Cancel ticket? (yes/no): ")
    if cancel.lower() == "yes":
        penalty = total * 0.20
        refund = total - penalty
        print("Cancellation Penalty:", round(penalty, 2))
        print("Refund Amount:", round(refund, 2))
    else:
        print("Final Fare:", round(total, 2))
```

C. Flowchart (text to copy and draw)

```
START
  |
Input passenger details
  |
Calculate distance-based fare
  |
Apply class multiplier
  |
Apply age concession
  |
Add reservation fee if applicable
  |
Calculate final fare
  |
Cancel ticket?
  |Yes: Calculate penalty and refund
```

```
  |No: Display final fare
  v
More passengers? --Yes--> Repeat
  |
 No
  v
STOP
```

D. Sample Output

```
Enter number of passengers: 1
Passenger 1
Enter passenger name: Rahul
Enter age: 65
Enter distance in km: 250
1. Sleeper
2. AC
3. First Class
Select class: 1

Passenger: Rahul
Ticket Fare: 260.0
Cancel ticket? (yes/no): no
Final Fare: 260.0
```

7. Retail Billing with Multiple Discount Rules

A. Algorithm

1. Start and input number of products.
2. For each product, input name, price, quantity, and category.
3. Calculate item amount and assign tax rate: Food 5%, Clothing 12%, Electronics 18%.
4. Add item amounts to subtotal.
5. If subtotal is at least Rs. 5,000, apply 10% discount, capped at Rs. 500.
6. Distribute discount proportionally across items and calculate tax on discounted item amounts.
7. Calculate final bill and print itemized invoice. Stop.

B. Python Code

```
products = []
subtotal = 0
n = int(input("Enter number of products: "))

for i in range(n):
    print("\nProduct", i + 1)
    name = input("Product name: ")
    price = float(input("Price: "))
    quantity = int(input("Quantity: "))
    category = input("Category (food/clothing/electronics): ")
    amount = price * quantity

    if category.lower() == "food":
        rate = 0.05
    elif category.lower() == "clothing":
        rate = 0.12
    elif category.lower() == "electronics":
        rate = 0.18
    else:
        print("Invalid category")
        continue

    subtotal += amount
    products.append([name, amount, rate])

if subtotal >= 5000:
    discount = min(subtotal * 0.10, 500)
else:
    discount = 0

tax_total = 0
print("\nITEMIZED INVOICE")
print("--------------------------------")
for name, amount, rate in products:
    discounted_amount = amount * (subtotal - discount) / subtotal
    tax = discounted_amount * rate
    tax_total += tax
    print(name, "Amount:", round(amount, 2),
          "Tax:", round(tax, 2))

final_bill = subtotal - discount + tax_total
print("--------------------------------")
print("Subtotal:", round(subtotal, 2))
print("Discount:", round(discount, 2))
print("Total Tax:", round(tax_total, 2))
print("Final Bill:", round(final_bill, 2))
```

C. Flowchart (text to copy and draw)

```
START
  |
Input number of products
  |
Input product details
  |
Calculate item amount and category tax rate
  |
Add amount to subtotal
  |
More products? --Yes--> Repeat
  |
 No
  v
Subtotal >= Rs. 5,000?
  |Yes: Apply 10% discount capped at Rs. 500
  |No: No discount
  v
Calculate tax on discounted amounts
```

```
  |
Calculate final bill
  |
Print itemized invoice
  |
STOP
```

D. Sample Output

```
Enter number of products: 2
Product 1
Product name: Shirt
Price: 1000
Quantity: 2
Category (food/clothing/electronics): clothing
Product 2
Product name: Mobile
Price: 4000
Quantity: 1
Category (food/clothing/electronics): electronics

ITEMIZED INVOICE
--------------------------------
Shirt Amount: 2000.0 Tax: 216.0
Mobile Amount: 4000.0 Tax: 648.0
--------------------------------
Subtotal: 6000.0
Discount: 500
Total Tax: 864.0
Final Bill: 6364.0
```

8. Manufacturing Cost and Profitability Analysis

A. Algorithm

1. Start and input product name, production quantity, material cost/unit, labour cost/unit, machine cost, wastage percentage, selling price/u
2. Calculate material cost and labour cost.
3. Calculate wastage cost as material cost × wastage percentage / 100.
4. Calculate total manufacturing cost.
5. Calculate revenue = quantity × selling price.
6. Calculate profit = revenue - total cost.
7. Calculate profit percentage = profit / total cost × 100.
8. Compare with target and display report. Stop.

B. Python Code

```
name = input("Enter product name: ")
quantity = int(input("Enter production quantity: "))
material = float(input("Material cost per unit: "))
labour = float(input("Labour cost per unit: "))
machine = float(input("Total machine cost: "))
wastage = float(input("Wastage percentage: "))
selling = float(input("Selling price per unit: "))
target = float(input("Required profit percentage: "))

material_cost = quantity * material
labour_cost = quantity * labour
wastage_cost = material_cost * wastage / 100
total_cost = material_cost + labour_cost + machine + wastage_cost
revenue = quantity * selling
profit = revenue - total_cost
profit_percentage = (profit / total_cost) * 100 if total_cost else 0

print("\nMANUFACTURING REPORT")
print("Product:", name)
print("Material Cost:", material_cost)
print("Labour Cost:", labour_cost)
print("Wastage Cost:", wastage_cost)
print("Machine Cost:", machine)
print("Total Cost:", total_cost)
print("Total Revenue:", revenue)
print("Profit:", profit)
print("Profit Percentage:", round(profit_percentage, 2), "%")

if profit_percentage >= target:
    print("Production meets profit target")
else:
    print("Production does not meet profit target")
```

C. Flowchart (text to copy and draw)

```
START
  |
Input production details
  |
Calculate material, labour, and wastage costs
  |
Calculate total manufacturing cost
  |
Calculate revenue and profit
  |
Calculate profit percentage
  |
Profit percentage >= target?
  |Yes: Meets target
  |No: Does not meet target
  v
Display manufacturing report
  |
STOP
```

D. Sample Output

```
Enter product name: Chairs
Enter production quantity: 100
Material cost per unit: 200
Labour cost per unit: 50
Total machine cost: 2000
Wastage percentage: 5
Selling price per unit: 350
Required profit percentage: 20

MANUFACTURING REPORT
```

Product: Chairs
Material Cost: 20000.0
Labour Cost: 5000.0
Wastage Cost: 1000.0
Machine Cost: 2000.0
Total Cost: 28000.0
Total Revenue: 35000.0
Profit: 7000.0
Profit Percentage: 25.0 %
Production meets profit target

9. Multi-Currency Travel Budget Analysis

A. Algorithm

1. Start and input travel budget in INR and number of expense categories.
2. For each category, input name, foreign-currency amount, and exchange rate to INR.
3. Convert amount to INR and calculate a 2% conversion fee.
4. Add converted amount and fee to category expense and total spending.
5. Identify the category with the largest total expense.
6. Compare total spending with budget.
7. Display remaining budget or excess amount and largest category. Stop.

B. Python Code

```
budget = float(input("Enter travel budget in INR: "))
n = int(input("Enter number of expense categories: "))
expenses = []
total = 0

for i in range(n):
    print("\nExpense", i + 1)
    category = input("Category name: ")
    amount = float(input("Expense in foreign currency: "))
    rate = float(input("Exchange rate to INR: "))

    converted = amount * rate
    fee = converted * 0.02
    final = converted + fee
    expenses.append([category, final])
    total += final

    print("Converted Amount:", round(converted, 2))
    print("Conversion Fee:", round(fee, 2))
    print("Total Expense:", round(final, 2))

largest = max(expenses, key=lambda x: x[1])
print("\nTRAVEL BUDGET REPORT")
print("Total Spending:", round(total, 2))
print("Largest Expense:", largest[0])
print("Expense Amount:", round(largest[1], 2))

if total <= budget:
    print("Remaining Budget:", round(budget - total, 2))
    print("Status: Within Budget")
else:
    print("Budget Exceeded:", round(total - budget, 2))
    print("Status: Over Budget")
```

C. Flowchart (text to copy and draw)

```
START
  |
Input budget and category count
  |
Input expense and exchange rate
  |
Convert to INR and calculate 2% fee
  |
Store category total and add to spending
  |
More categories? --Yes--> Repeat
  |
 No
  v
Find largest expense category
  |
Compare spending with budget
  |
Within budget? --Yes--> Show remaining budget
  |No--> Show excess amount
  v
Display travel report
  |
STOP
```

D. Sample Output

```
Enter travel budget in INR: 100000
Enter number of expense categories: 2
Expense 1
Category name: Hotel
Expense in foreign currency: 300
```

```
Exchange rate to INR: 85
Converted Amount: 25500.0
Conversion Fee: 510.0
Total Expense: 26010.0
Expense 2
Category name: Shopping
Expense in foreign currency: 500
Exchange rate to INR: 85
Converted Amount: 42500.0
Conversion Fee: 850.0
Total Expense: 43350.0

TRAVEL BUDGET REPORT
Total Spending: 69360.0
Largest Expense: Shopping
Expense Amount: 43350.0
Remaining Budget: 30640.0
Status: Within Budget
```

10. Campus Event Financial Planning

A. Algorithm

1. Start and input event name, venue, equipment, transport, catering/person, and certificate/person costs.
2. Input sponsorship, registration fee/person, and actual participants.
3. Calculate fixed cost = venue + equipment + transport.
4. Calculate actual variable cost = participants × (catering + certificate).
5. Calculate total expense and total funds = sponsorship + registration collections.
6. Calculate contribution per participant = registration fee - variable cost per participant.
7. If contribution is positive, calculate minimum break-even participants, accounting for sponsorship.
8. Compare total funds with total expense and display surplus or shortfall. Stop.

B. Python Code

```
event = input("Enter event name: ")
venue = float(input("Venue cost: "))
equipment = float(input("Equipment cost: "))
catering = float(input("Catering cost per participant: "))
transport = float(input("Transport cost: "))
certificate = float(input("Certificate cost per participant: "))
sponsorship = float(input("Enter sponsorship amount: "))
fee = float(input("Registration fee per participant: "))
participants = int(input("Enter actual participants: "))

fixed_cost = venue + equipment + transport
variable_cost = participants * (catering + certificate)
total_expense = fixed_cost + variable_cost
registration = participants * fee
total_funds = sponsorship + registration

contribution = fee - catering - certificate
if contribution > 0:
    required = max(0, (fixed_cost - sponsorship + contribution - 1)
                   // contribution)
else:
    required = None

print("\nCAMPUS EVENT FINANCIAL REPORT")
print("Event:", event)
print("Fixed Cost:", fixed_cost)
print("Variable Cost:", variable_cost)
print("Total Expense:", total_expense)
print("Sponsorship:", sponsorship)
print("Registration Collection:", registration)
print("Total Funds:", total_funds)

if required is not None:
    print("Minimum Participants to Break Even:", int(required))
else:
    print("Break-even is not possible with the given fee")

if total_funds >= total_expense:
    print("Event Status: Financially Successful")
    print("Surplus:", total_funds - total_expense)
else:
    print("Event Status: Financial Shortfall")
    print("Shortfall:", total_expense - total_funds)
```

C. Flowchart (text to copy and draw)

```
START
  |
Input event details and costs
  |
Calculate fixed and variable costs
  |
Calculate total expenses
  |
Input sponsorship and registration fee
  |
Calculate break-even participant count
  |
Input actual participants
  |
Calculate registration collections and total funds
  |
Compare funds with expenses
  |
Funds >= expenses?
  |Yes: Display surplus
  |No: Display shortfall
```

```
  v
Display event financial report
  |
STOP
```

D. Sample Output

```
Enter event name: Tech Fest
Venue cost: 10000
Equipment cost: 5000
Catering cost per participant: 100
Transport cost: 3000
Certificate cost per participant: 20
Enter sponsorship amount: 5000
Registration fee per participant: 200
Enter actual participants: 100

CAMPUS EVENT FINANCIAL REPORT
Event: Tech Fest
Fixed Cost: 18000.0
Variable Cost: 12000.0
Total Expense: 30000.0
Sponsorship: 5000.0
Registration Collection: 20000.0
Total Funds: 25000.0
Minimum Participants to Break Even: 93
Event Status: Financial Shortfall
Shortfall: 5000.0
```